SOLAR

From a seven-agent ensemble to label-efficient evaluation and oversight of detectors and LLM judges

End-to-end project report: foundations, Phases 1 to 5, tools, flaws found and results

Prepared for Agnes Teresa Davuri
October 14, 2026

What this document is. A complete record of the project: what each phase asked, what data, code and compute it used, what happened, what was gained, and what failed. Every number comes from a run in this project. Every success bar was fixed in the code before the run it judged. Negative results are reported with the same weight as positive ones.

Contents

Executive summary 3

How the project fits together 4

Part A. Foundations (before Phase 1) 5

A1. Auditing the original SOLAR notebook . . . 5

A2. Data inventory and audit . . . 5

A3. Do detectors transfer? Cross-dataset baselines . . . 7

A4. Estimating the bot share from a detector (the BotShare study) . . . 9

Phase 1: do human-labeled samples give valid numbers? 11

Phase 2: does an LLM labeler earn its place? 14

Phase 3: can a pilot choose which scorer to trust? 17

Phase 4: how often does an LLM judge disagree with humans? 19

Step 1: judge reliability on MT-Bench . . . 19

Step 1b: a second judge as a risk score . . . 21

Phase 5: can we monitor continuously? 23

Tools and artifacts built 26

Flaws found along the way and how they were fixed 26

Scorecard and lessons 28

Limitations and threats to validity 29

Related work 29

Reproducibility 30

Glossary 31

Appendix A. Bot-share estimates per dataset 31

Appendix B. Text transfer results 32

Appendix C. Single-feature AUC for every feature and dataset 32

Executive summary

Question. Can you trust a bot detector, a misinformation classifier or an LLM judge on your own data, and if not, how do you find out cheaply? The project began as a seven-agent ensemble for bot and misinformation detection and ended as a set of tested methods and two browser tools for answering that question with human labels.

Answer in one paragraph. Automatic scores do not carry over to new data, and standard corrections do not fix that. A small randomly chosen human-labeled sample does: it gives intervals that cover the truth as promised, with about ten times less error than counting what a detector flags. A short pilot can choose which score to trust without risking validity, intervals can be checked continuously without breaking their guarantee, and the same machinery measures how often an LLM judge disagrees with humans.

Headline results

- **Detectors do not transfer.** Cross-dataset AUC fell 0.11 to 0.27 on six of seven bot datasets. At a 5% false-flag rate a detector caught only 3.5% to 37% of bots on data it had not seen. Text detectors scored 0.68 to 1.0 on their own data and 0.47 to 0.65 elsewhere.
- **Bot-share estimation from a detector fails.** Counting flagged accounts under-counted the true share by about 29 points; adjusted counts jumped to 0% or 100%; nothing beat “always guess the average”. Stable features and simpler models did not rescue it.
- **A labeled sample works.** At 200 labels the interval half-width is about 5.4 points and the typical error 2.7 points (counting detector flags: 28.9). Coverage was 0.93 to 0.96 across ten datasets.
- **An LLM labeler did not earn its place.** A 3B model was at chance on LIAR and strong on CoAID (AUC 0.87); the best LLM design was 5.9% narrower than the best non-LLM design against a 15% bar. Rubric prompts did not help overall.
- **A pilot can pick the scorer safely.** Coverage 0.95, about 6% narrower than the default design, never wider than random sampling. The gain is in-sample.
- **LLM-judge reliability is measurable.** GPT-4 disagrees with the human majority on 28.7% of 1,612 MT-Bench items (22.7% humanities to 37.3% coding), known to about ±5.9 points from 200 labels. A second local judge failed as a risk score (AUC 0.552).
- **Monitoring can be continuous.** A confidence sequence missed the truth at some look in 2% to 3% of simulated streams, against 42% to 46% for ordinary intervals checked after every label, at about 1.5 to 1.6 times the width.

The full scorecard, with the evidence behind each verdict, is in the section “Scorecard and lessons”.

How the project fits together

The work moved in stages, and each stage was decided by the evidence from the one before. The table is the map; later sections give the details.

Stage	Question it asked	What was done	Compute	Outcome
Foundations A	Do the original claims hold up?	Audited the seven-agent notebook: ablation, fallbacks, compute claim, FakeNewsNet	notebook outputs	Most headline claims unsupported
Foundations B	What data do we have, and is it clean?	Loaded 7 bot, 4 text datasets; audited overlap, duplicates, shortcuts	CPU	3,351 shared accounts; conflicting duplicates
Foundations C	Do detectors transfer?	Leave-one-dataset-out bot detectors; text train-A test-B	CPU	No: AUC drops 0.11 to 0.27
Foundations D	Can a detector estimate the bot share?	Five quantification methods, a deployable API, a rescue experiment	CPU	No: nothing beat the trivial guess
Phase 1	Do human-labeled samples give valid numbers?	Stratified and PPI estimators, coverage simulation on 7 datasets	CPU	Yes: coverage 0.94 to 0.955
Phase 2	Does an LLM labeler earn its place?	Qwen2.5-3B scores 3 text datasets, 8 sampling arms	T4 GPU	No: 5.9% vs 15% bar
Phase 3	Can a pilot choose the scorer?	Pilot-then-stratify planner on the cached scores	CPU	Yes (in-sample): about 6% narrower
Phase 4 (1, 1b)	How often does an LLM judge err?	MT-Bench: GPT-4 vs human experts; second local judge	CPU, T4 GPU	Validity yes; second judge no
Phase 5	Can we monitor continuously?	Betting confidence sequences and a drift alarm	CPU	Yes: 2% to 3% miss vs 42% to 46%
Tools and docs	Can a team use it?	Two browser tools, a results report, a write-up	none	Built; tested on synthetic data and screenshots

Conventions used throughout

- **AUC**: probability that a randomly chosen positive scores above a randomly chosen negative. 0.5 is chance, 1.0 is perfect.
- **Coverage**: how often a 95% interval contains the true value across repeated simulated samples. About 95% is the promise.
- **Half-width**: half the interval's width, quoted in percentage points; "±5.4 points" means an interval 10.8 points wide.
- **The human** in every simulation is the dataset's own true label. This is the best case: real reviewers make errors.
- **Bar fixed in advance**: the pass/fail criteria were written into each script's docstring before its real run. They were not registered with any outside body.

Part A. Foundations (before Phase 1)

A1. Auditing the original SOLAR notebook

What was claimed. A seven-agent ensemble (a fast cascade, source credibility, fact grounding, adversarial robustness, a stacked ensemble, metadata profiling and a zero-shot LLM agent) fused by entropy-weighted voting, reporting F1 0.983 on Cresci-2017, a cascade that "resolves 68.5% of samples", and strong results on FakeNewsNet.

What the audit found.

- **Fusion added almost nothing.** The fused F1 was 0.9831; the best single agent alone scored 0.9826, and the best classical baseline (an SVM) 0.9648.
- **Four of seven agents contributed nothing.** Removing the first agent, the adversarial agent or the LLM agent changed F1 by exactly 0.0; removing the source-credibility agent slightly improved it. Only three agents carried the signal, so "each agent contributes uniquely" was unsupported.
- **The LLM agent never ran.** All 200 of its responses were API fallbacks.
- **The "68.5% compute saved" figure was not a measurement.** It was the share of cases where the first agent was confident; the fusion step still ran every agent, and no timing was recorded.
- **FakeNewsNet was at chance.** Every model scored 0.42 to 0.48 accuracy, validation AUC was 0.19 for the first agent, and the cross-dataset table gave one identical F1 (0.3359) for all models. The "state of the art" claim could not stand. Later work traced part of this to duplicate articles carrying both labels.
- **The Cresci-2017 numbers were not comparable to the full benchmark.** The notebook used 4,465 accounts at 22% bots; the full benchmark has 14,368 accounts at 75.8% bots.
- **One agent's network was simulated** from follower-count similarity rather than real edges.

> **What was gained.** A refusal to trust headline numbers, and the habit that carried through the rest of the project: ablate every component, fix the success bar before the run, and report failures. It also moved the project away from the planetary agents as its headline and toward evaluation.

A2. Data inventory and audit

Three families of data were used. All were loaded from public datasets attached on Kaggle; the bot datasets are in the TwiBot-22 format (profile fields, labels and official splits).

Dataset	Items	Positive share	Notes
botometer-feedback-2019	518	26.6%	bot accounts
cresci-2015	5,301	63.2%	all 3,351 bots also in cresci-2017
cresci-2017	14,368	75.8%	bot accounts
cresci-rtbust-2019	693	50.9%	bot accounts
cresci-stock-2018	13,276	53.5%	bot accounts
gilani-2017	2,484	43.9%	bot accounts
midterm-2018	50,538	84.0%	22% of rows share identical profile features
LIAR	12,836	44.2%	political claims; positive = rated false, pants-on-fire or barely true
Fake and Real News	38,802	46.1%	kept after dropping 6,096 empties and near-duplicates; dateline and subject removed
CoAID	5,660	16.7%	four dated folders pooled and de-duplicated (5,975 rows before)
FakeNewsNet	175	50.0%	of 422 rows, 133 duplicates; 114 of about 289 unique articles under both labels
MT-Bench judgments	1,612	28.7%	items with a human majority and a GPT-4 verdict; positive = they disagree

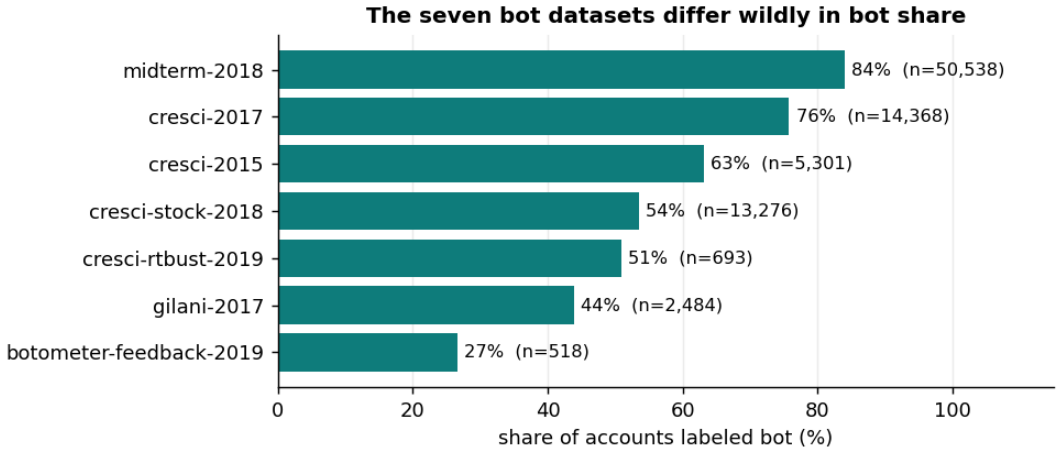


Figure 1. Bot share runs from 27% to 84% across the seven datasets, so a detector-only count cannot be compared between them.

Audit findings

Finding	Detail	Consequence
Shared accounts	All 3,351 bots of cresci-2015 are also in cresci-2017 (63% of the smaller dataset); the other pairs share 1 to 3 accounts	Cross-dataset training must drop shared accounts: cresci-2015 first scored 0.986 and fell to 0.877
Identical profiles	11,217 of 50,538 midterm-2018 rows (22%) and 154 cresci-2017 rows share identical feature vectors	Suggests templated accounts; harmless for design-based intervals
Conflicting duplicates	FakeNewsNet: 114 of about 289 unique articles appear under both labels (generic landing pages, claims reused across labels)	Text cannot determine the label for those items; the dataset was cut to 175 articles and used only as a small check
Label-leaking columns	Fake and Real News: a Reuters dateline in one class, and subject and date columns	Removed; the dataset is still very easy (within-dataset AUC 0.999)
Length shortcut	Text length alone gives AUC 0.645 on CoAID (news and short claims mixed); 0.51 to 0.53 elsewhere	CoAID news analysed separately; text models must beat 0.645 to prove they read anything
Top single feature	One column nearly decides some datasets: followers in midterm (AUC 0.029), tweets in cresci-2017 (0.068), ratio in cresci-2015 (0.009)	Within-dataset scores flatter the detector; see the next section

A3. Do detectors transfer? Cross-dataset baselines

Method. Bot detector: gradient-boosted trees on 13 profile features computed identically for every dataset (log followers, following, tweets and listed counts, the followers-to-following ratio, verified flag, description present and length, URL and location present, name and username length, username digit ratio). Account age was excluded because collection dates differ by dataset. For each dataset the model trains on the other six (at most 4,000 accounts per source, accounts shared with the held-out dataset removed) and is tested on the whole held-out dataset. Text detector: TF-IDF with logistic regression, trained on one dataset and tested on another.

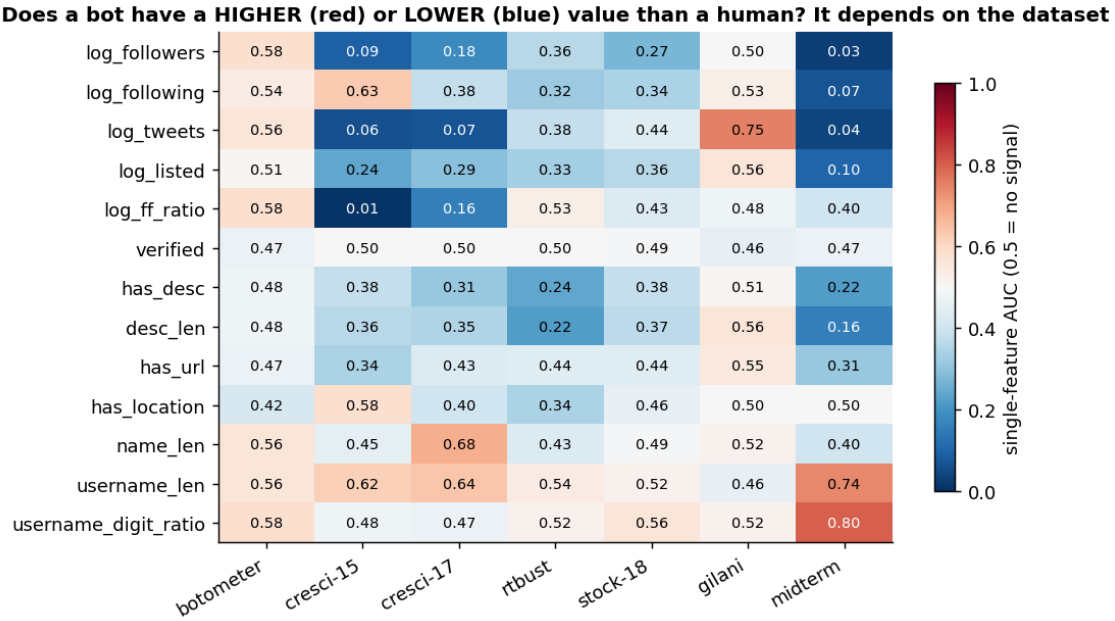


Figure 2. Single-feature AUC by dataset. Below 0.5 (blue) bots have lower values than humans, above 0.5 (red) higher. Bots tweet far less than humans in three datasets and more in gilani: "bot" is not one kind of account.

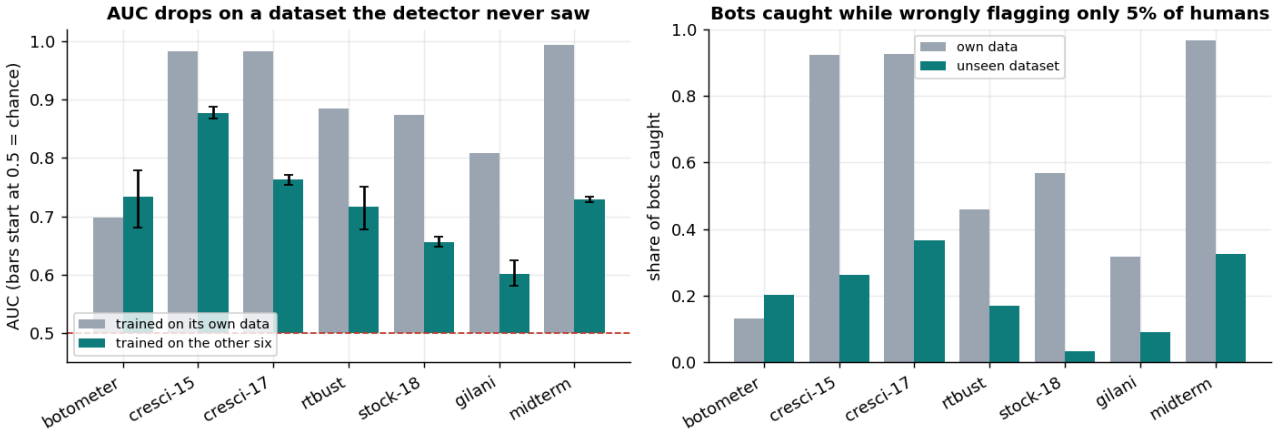


Figure 3. Left: AUC when trained on the dataset's own split (grey) against trained on the other six (teal, with 95% interval). Right: share of bots caught while wrongly flagging only 5% of humans.

Held-out dataset	Accounts	Own-data AUC	Unseen AUC (95% CI)	AUC drop	Bots caught at 5% false flags: own / unseen
botometer-feedback-2019	518	0.698	0.733 [0.681, 0.779]	-0.035	13.3% / 20.3%
cresci-2015	5,301	0.982	0.877 [0.867, 0.887]	+0.105	92.3% / 26.3%
cresci-2017	14,368	0.983	0.763 [0.754, 0.771]	+0.220	92.6% / 36.8%
cresci-rtbust-2019	693	0.884	0.716 [0.678, 0.751]	+0.168	45.9% / 17.0%
cresci-stock-2018	13,276	0.874	0.656 [0.648, 0.665]	+0.218	56.8% / 3.5%
gilani-2017	2,484	0.809	0.601 [0.582, 0.624]	+0.208	31.8% / 9.2%
midterm-2018	50,538	0.993	0.729 [0.724, 0.734]	+0.265	96.7% / 32.6%

Five of seven datasets lose 0.16 to 0.27 AUC; botometer-feedback shows no drop but its own-data score rests on only 53 test accounts. At a 5% false-flag rate the unseen-data detector catches as few as 3.5% of bots (stock-2018).

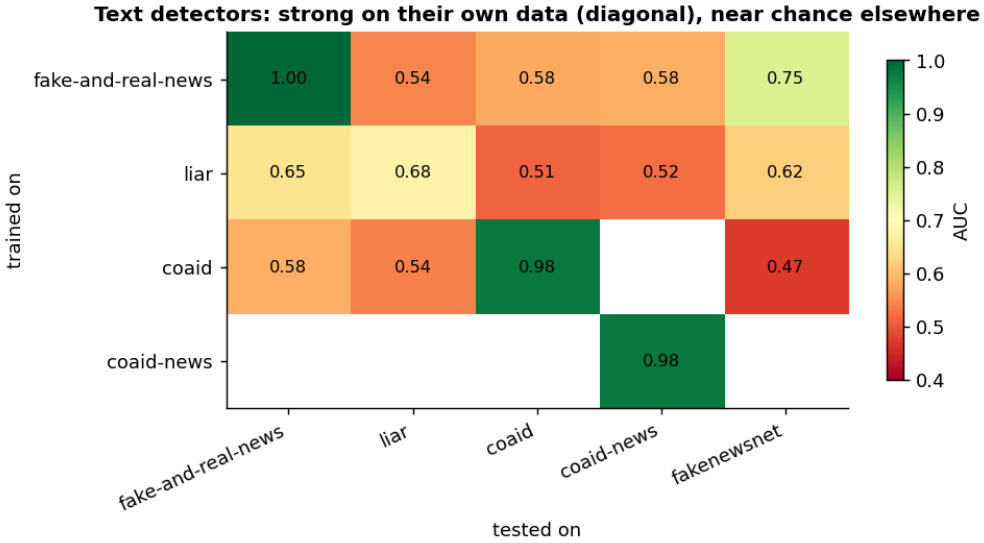


Figure 4. Text detectors: AUC when trained on the row dataset and tested on the column dataset. The diagonal (own data) is strong; everything else is near chance.

LIAR is the only honest text benchmark here: a 0.675 AUC on itself with no shortcut. Fake and Real News and CoAID score 0.98 to 0.999 within themselves and 0.51 to 0.65 across, because the labels

describe different things (outlet type versus fact-checked claims).

> **What was gained.** A clear and quantified answer to “do detectors transfer”: no. Also a methodological safeguard, leave-one-dataset-out evaluation with shared accounts removed, which every later phase reuses.

A4. Estimating the bot share from a detector (the BotShare study)

Idea. If a detector cannot label accounts reliably, perhaps it can estimate the *share* of bots in a new group, using standard quantification methods: classify-and-count (CC), average score (PCC), adjusted count (ACC), adjusted average (PACC) and the EM prior-adjustment method (SLD). Error rates for the corrections were measured on cross-dataset scores so they would reflect real dataset shift. Two feature profiles were trained: *full* (13 features) and *core* (followers, following, tweets and their ratio) so an upload with missing columns would be served by a model validated for it. A FastAPI app and a front-end were built around it, aggregate-only and in memory.

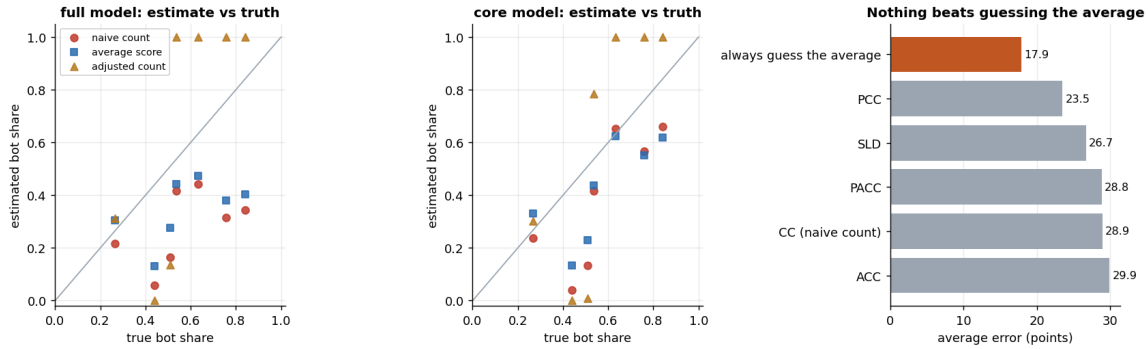


Figure 5. Estimated against true bot share for three estimators (dashed diagonal = perfect) and the average error of each method for the full model.

Method	Full model: average error (points)	Core model: average error (points)
Always guess the training average	17.9	17.9
Average score (PCC)	23.5	16.9
Naive count (CC)	28.9	18.8
EM prior adjustment (SLD)	26.7	27.5
Adjusted average (PACC)	28.8	29.4
Adjusted count (ACC)	29.9	28.5

The corrections assume the detector’s error rates carry over to new data. They do not: ACC, PACC and SLD saturated at 0% or 100% on most datasets. Naive counting had a bias of −28.9 points. The core model’s PCC beat the guess by about one point, but on only 4 of 7 datasets, which is noise; the first reliability rule (strictly lower error) would have wrongly called it reliable, so the rule was tightened to require a 25% lower error and wins on at least 70% of datasets. The app now withholds its headline number unless that bar is met.

The rescue attempt

Hypothesis: extra features encode how each dataset was collected, so keep only features whose bot-versus-human direction is the same across training datasets (chosen inside each fold, with no leak), with simpler or monotone models. The bar, fixed in advance: error at most 75% of the trivial guess and a win on 5 of 7 datasets.

Variant	Best method	Error (points)	Naive count error	Guess-the-average error	Beats guess on	Verdict
A boosted trees, all 13	PCC	23.5	28.9	17.9	1/7	fail
B boosted trees, core 4	PCC	16.9	18.8	17.9	4/7	fail
C boosted trees, stable	PCC	21.6	33.6	17.9	2/7	fail
D stable + monotone	PCC	20.2	27.7	17.9	3/7	fail
E logistic, stable	PCC	20.3	26.5	17.9	2/7	fail

No variant passed. The feature table (Figure 2) explains why: nearly every informative feature changes sign between datasets, so almost nothing is stable. No estimator or correction can recover the share when the profile of a bot changes from dataset to dataset.

> **What was gained.** A firm negative result, and the pivot that shaped the rest: since the model cannot tell you the answer, ask a human for a small, randomly chosen sample and measure how far that can be trusted. Hand-labeling about 200 random accounts gives about ±7 points, better than any detector-based estimate tested (17 to 29 points).

Phase 1: do human-labeled samples give valid numbers?

Question. Given a detector and a limited human-labeling budget, can we get prevalence, precision and recall with intervals that really cover the truth about 95% of the time, and where should the labels go?

What we used

- **Data:** the seven bot datasets (87,178 accounts).
- **Detector:** the transferred bot detector from Section A3 (each dataset scored by a model that never saw it; AUC 0.60 to 0.88), so the experiment mimics deploying a detector on unfamiliar data.
- **Code:** the *solar* package: audit.py, plan.py (designs), estimate.py (estimators), detector.py, simulate.py (coverage experiment) and run_phase1.py, with synthetic-data tests.
- **Settings:** three designs, budgets of 100, 200 and 400 labels, 300 repetitions per cell, five equal-count strata by detector score, budgets above half a population skipped. The dataset's true label played the human reviewer.

What we did

- **Designs.** Simple random sampling; equal allocation across five score strata; Neyman allocation (labels in proportion to stratum size times expected spread, guessed from the scores).
- **Labeled-only estimator.** The stratum-weighted mean of the human labels with a finite-population correction and a 95% Wald interval. Precision divides by the exactly known number of flagged items. Recall is a ratio with a delta-method interval.
- **Prediction-powered inference (PPI).** Adds the detector's population-wide mean and corrects it with the labeled residuals, with a weight chosen to minimise variance. It is unbiased for any detector, even a bad one.
- **Coverage experiment.** For each dataset, design and budget, draw many samples, estimate, and record how often the interval contains the truth, its width, and its error, against the error of counting what the detector flags.

> **Bar fixed in advance (n = 200):** (1) pooled coverage of the prevalence and precision intervals within 0.92 to 0.98; (2) no dataset's prevalence coverage below 0.88; (3) prevalence error at most 50% of the detector-only count's error; (4) PPI shipped over labeled-only only if it narrows the interval by at least 5%.

Results

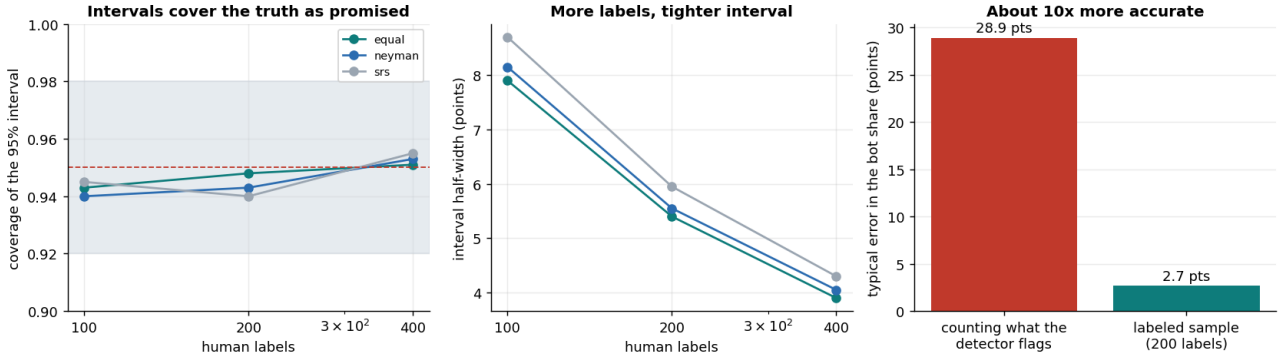


Figure 6. Left: coverage stays near the 95% target (shaded band 0.92 to 0.98). Middle: half-width falls as labels grow. Right: at 200 labels the typical error is about ten times smaller than counting detector flags.

Design	Labels	Method	Coverage	Lowest dataset coverage	Interval width	Typical error	Error of detector count
equal	100	labeled	0.943	0.913	0.158	0.041	0.289
equal	100	ppi	0.941	0.917	0.157	0.041	0.289
equal	200	labeled	0.948	0.933	0.108	0.027	0.289
equal	200	ppi	0.946	0.927	0.108	0.027	0.289
equal	400	labeled	0.951	0.937	0.078	0.020	0.325
equal	400	ppi	0.951	0.933	0.077	0.020	0.325
neyman	100	labeled	0.940	0.923	0.163	0.042	0.289
neyman	100	ppi	0.940	0.923	0.162	0.042	0.289
neyman	200	labeled	0.943	0.930	0.111	0.029	0.289
neyman	200	ppi	0.943	0.930	0.111	0.029	0.289
neyman	400	labeled	0.953	0.927	0.081	0.020	0.325
neyman	400	ppi	0.953	0.927	0.081	0.020	0.325
srs	100	labeled	0.945	0.920	0.174	0.044	0.289
srs	100	ppi	0.933	0.917	0.162	0.042	0.289
srs	200	labeled	0.940	0.923	0.119	0.031	0.289
srs	200	ppi	0.948	0.940	0.110	0.029	0.289
srs	400	labeled	0.955	0.940	0.086	0.022	0.325
srs	400	ppi	0.953	0.933	0.080	0.020	0.325

Table: prevalence results for all designs, budgets and methods, averaged across the seven datasets (widths and errors as fractions; 0.108 means an interval 10.8 points wide). The 400-label rows leave out the two smallest datasets.

Verdict at 200 labels against the bar

Design	Method	Prevalence coverage	Lowest dataset	Precision coverage	Width	Error vs detector count	PPI width gain	Pass
equal	labeled	0.948	0.933	0.941	0.108	0.10		yes
equal	ppi	0.946	0.927	0.940	0.108	0.10	0.4%	yes
neyman	labeled	0.943	0.930	0.939	0.111	0.10		yes
neyman	ppi	0.943	0.930	0.939	0.111	0.10	0.3%	yes
srs	labeled	0.940	0.923	0.942	0.119	0.11		yes
srs	ppi	0.948	0.940	0.934	0.110	0.10	7.0%	yes

What the numbers say

- **Validity held.** Coverage was 0.933 to 0.955 in every cell and the lowest single dataset was 0.913 (at 100 labels).
- **Accuracy.** At 200 labels the typical error was 2.7 points (interval half-width about 5.4) against 28.9 points for counting what the detector flags: about ten times better.
- **The detector helps a little.** Stratifying on a weak detector's score narrowed the interval about 9% against random sampling (0.108 against 0.119).
- **PPI added almost nothing on top of strata** (0.3% to 0.4%) and 7% only under random sampling, and even then it did not beat stratified labeled-only. Neyman allocation was no better than equal (0.111 against 0.108) because its guess about uncertainty comes from the same weak scores.

Flaws and corrections

A first test expected PPI to always narrow the interval when the detector is good. It did not, for stratified designs, and the diagnostic showed why: stratifying on the score already uses the detector's information, so a detector's information can go into the design or into the estimator, and using both adds almost nothing. The test was corrected to encode what is true. Separate synthetic tests confirmed the property that makes the method safe: intervals stayed valid even when the detector was useless, miscalibrated or inverted.

> **Verdict: passed.** Ship equal-strata sampling with the labeled-only estimator; PPI was dropped. **What was gained:** a validated way to measure a detector with about 200 labels, the insight that sample size, not detector quality, drives accuracy, and a safe fallback: a bad scorer costs width, never validity.

Phase 2: does an LLM labeler earn its place?

Question. Phase 1 showed the weak detector barely narrows intervals. An LLM's information is different from a bag-of-words detector's. Can an LLM scorer, and a prompt that states the labeling definition explicitly (the "Definition agent"), narrow intervals at the same human budget?

What we used

- **Model:** Qwen2.5-3B-Instruct (open weights, fp16) on a Kaggle T4 GPU through transformers; no API key. The score is the probability of "Yes" against "No" read from the next-token logits, so no text is generated.
- **Data:** LIAR, Fake and Real News and CoAID; a random population of 4,000 items per dataset (seed 42); texts truncated to 1,200 characters; 24,000 prompts in all (3 datasets, 4,000 items, 2 prompt types).
- **Prompts:** *generic* ("Is the text above misinformation (false or misleading)? Answer Yes or No") and *rubric*, which states what "positive" means for that dataset and tells the model to judge only by it. The definitions were: LIAR, a claim fact-checkers rated false, pants-on-fire or mostly false; CoAID, a COVID-19 news item or claim found false or misleading; Fake and Real News, an article from a fabricated or highly unreliable outlet as opposed to wire-style reporting.
- **Baseline detector:** TF-IDF with logistic regression, trained on the other text datasets.
- **Code:** labelers.py (three backends: local model, Claude API, mock), prompts.py, text_data.py, simulate2.py, run_phase2.py; scores cached so a run can resume.

What we did

A smoke test (300 items per dataset) ran first and confirmed the model downloads, scores and the pipeline completes; it was only indicative, because 150 labels were half the 300-item population and the finite-population effect squeezed every arm to nearly the same width. The real run then compared eight sampling arms at equal human budget, 300 repetitions per cell:

- random sampling, labeled only
- strata on TF-IDF, labeled only (the Phase 1 default)
- strata on the LLM score (generic prompt), labeled only
- strata on the LLM score (rubric prompt), labeled only
- random sampling plus PPI with the LLM (rubric)
- strata on TF-IDF plus PPI with the LLM (generic, and rubric)
- strata on the LLM plus PPI with the same LLM (rubric)

> **Bar fixed in advance (n = 200):** (1) an arm counts only with coverage within 0.92 to 0.98; (2) the LLM earns its place only if its best valid arm is at least 15% narrower (pooled) than the best non-LLM arm and at least 10% narrower on two or more datasets; (3) the Definition agent earns its place only if the rubric prompt beats the generic prompt by at least 0.02 AUC pooled and its stratified arm is at least 5% narrower.

Results

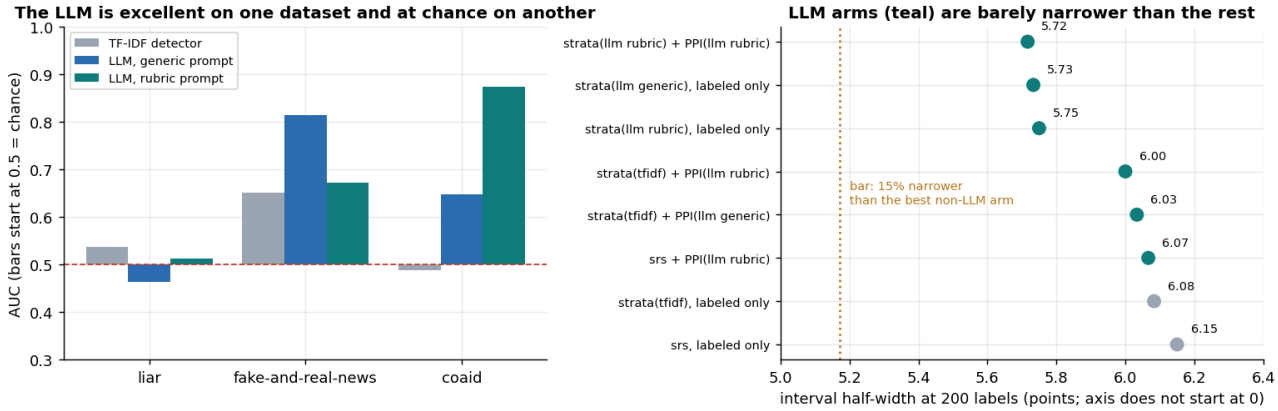


Figure 7. Left: AUC of each scorer against the truth (bars start at 0.5, chance). Right: interval half-width at 200 labels for each arm; the dotted line is the 15% bar.

Dataset	TF-IDF AUC	LLM generic	LLM rubric	Correlation of LLM with TF-IDF (generic / rubric)
LIAR	0.537	0.464	0.513	0.017 / 0.043
Fake and Real News	0.651	0.814	0.672	0.026 / 0.086
CoAID	0.487	0.647	0.874	0.005 / 0.013

Arm (n = 200)	CoAID	Fake and Real News	LIAR	Mean width	Coverage
strata(llm rubric) + PPI(llm rubric)	0.081	0.128	0.134	0.114	0.951
strata(llm generic), labeled only	0.098	0.112	0.134	0.115	0.949
strata(llm rubric), labeled only	0.082	0.129	0.134	0.115	0.943
strata(tfidf) + PPI(llm rubric)	0.098	0.128	0.134	0.120	0.946
strata(tfidf) + PPI(llm generic)	0.101	0.128	0.133	0.121	0.944
srs + PPI(llm rubric)	0.098	0.132	0.134	0.121	0.959
strata(tfidf), labeled only	0.101	0.130	0.134	0.122	0.949
srs, labeled only	0.100	0.135	0.134	0.123	0.938

LLM arm	Coverage	Width	Narrower than best non-LLM	Datasets 10% or more narrower	Valid	Earns its place
random sampling + PPI(rubric)	0.959	0.121	0.2%	0	yes	no
strata(LLM generic), labeled	0.949	0.115	5.6%	1	yes	no
strata(LLM rubric) + PPI(rubric)	0.951	0.114	5.9%	1	yes	no
strata(LLM rubric), labeled	0.943	0.115	5.4%	1	yes	no
strata(TF-IDF) + PPI(generic)	0.944	0.121	0.6%	0	yes	no
strata(TF-IDF) + PPI(rubric)	0.946	0.120	1.2%	0	yes	no

What the numbers say

- **Quality varies wildly by dataset.** On LIAR the LLM is at chance (0.46 to 0.51): checking political claims needs outside knowledge a small model without retrieval lacks. On CoAID the rubric prompt reaches 0.874. On Fake and Real News the rubric prompt *hurt* (0.672 against 0.814 generic).
- **The information is complementary but little of it was used.** The LLM's errors are nearly uncorrelated with TF-IDF (correlation 0.01 to 0.09), yet PPI on top of TF-IDF strata gained only about 1%. Using the LLM score as the stratifier did much better where it was accurate (CoAID: 0.082 against 0.101, about 19% narrower). A plausible reason, not tested, is that the LLM's probabilities are poorly calibrated for a linear correction.
- **Coverage stayed 0.94 to 0.96 for every arm**, including an LLM at chance on LIAR: a bad scorer costs width, never validity.
- **The Definition agent changed widths by −0.2%.** The rubric raised AUC by 0.045 pooled, but the gain on CoAID (+0.23) was offset by the loss on Fake and Real News (−0.14).

Verdict: failed both bars. The best LLM arm was 5.9% narrower (bar 15%) and on one dataset only (bar: two). **What was gained:** a working open-model labeling pipeline that needs no API key, proof that safe use of an LLM score is possible, and the observation that scorer quality is dataset-specific, which motivated Phase 3.

Phase 3: can a pilot choose which scorer to trust?

Question. Phase 2 showed no fixed choice of scorer is safe: the LLM is excellent on one dataset and useless on another, and the rubric helps one and hurts another. Could a small random pilot measure each candidate on *this* data, then spend the rest of the budget on the best one, with random sampling as the fallback?

What we used

- **Data and scores:** the cached Phase 2 scores for the three text datasets (TF-IDF, LLM generic, LLM rubric). No GPU and no model needed.
- **Code:** simulate3.py (the planner, estimator and bar), run_phase3.py, tests including unbiasedness and the safe-fallback check.
- **Settings:** 200 labels of which a pilot of 60; five strata; 300 repetitions per cell; a scorer is picked only if its pilot AUC is at least 0.6; otherwise the remaining labels use random sampling.

Design and estimator

The pilot is a random sample of all items, so its mean is an unbiased estimate (A). Stage two stratifies the remaining items by the chosen scorer and estimates the prevalence of the remainder; adding the pilot's known labels gives a second unbiased estimate (B). The two have uncorrelated noise, so they are combined with **fixed weights computed from the sample sizes and the population size only**, which keeps the combination exactly unbiased: $w = f_B / (f_A + f_B)$, where $f_A = (1 - m/N) / m$ and $f_B = ((N - m)/N)^2 (1 - (n - m)/(N - m)) / (n - m)$; m is the pilot size, n the total labels, N the population size.

Bar fixed in advance (n = 200, pilot 60): (1) pooled coverage within 0.92 to 0.98 and no dataset below 0.90; (2) at least 5% narrower than the Phase 1 default, strata on TF-IDF; (3) never more than 3% wider than random sampling on any dataset.

Results

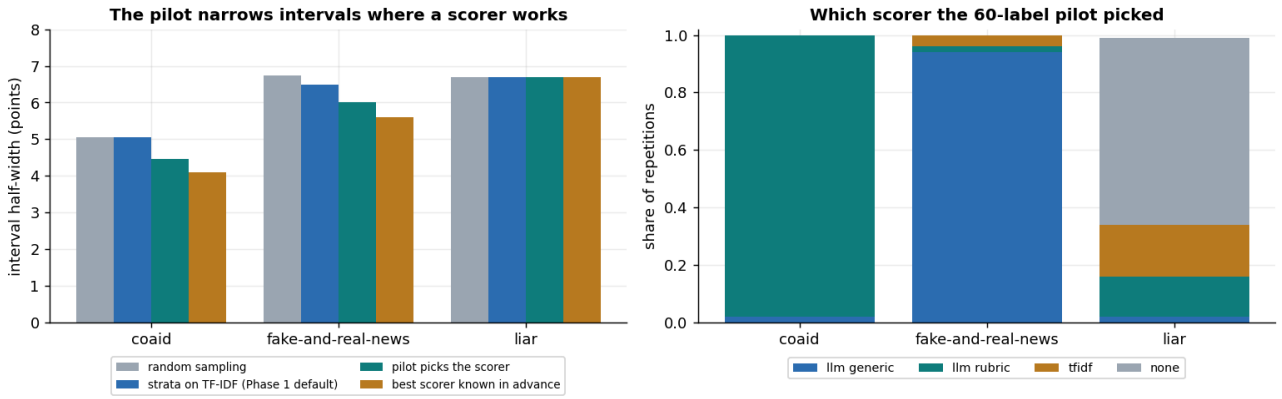


Figure 8. Left: interval half-width at 200 labels for four designs (bars start at zero). Right: which scorer the 60-label pilot picked, as a share of repetitions.

Dataset	Random sampling	Strata on TF-IDF	Pilot picks the scorer	Best scorer known in advance
CoAID	0.101	0.101	0.089	0.082
Fake and Real News	0.135	0.130	0.120	0.112
LIAR	0.134	0.134	0.134	0.134
Pooled mean	0.1233	0.1217	0.1143	0.1093

Dataset	LLM generic	LLM rubric	TF-IDF	None (random sampling)
CoAID	2%	98%	0%	0%
Fake and Real News	94%	2%	4%	0%
LIAR	2%	14%	18%	65%

- **Validity:** pooled coverage 0.95, lowest dataset 0.927.
- **Gain:** about 6% narrower than the Phase 1 default (bar: 5%) and 7.3% narrower than random sampling. On CoAID 12% and on Fake and Real News 11% narrower than random sampling; unchanged on LIAR.
- **Safety:** never wider than random sampling (worst dataset −0.1%).
- **Selection:** with 60 labels the pilot picked the best scorer 98% of the time on CoAID and 94% on Fake and Real News, and mostly picked nothing on LIAR, where nothing beats chance.
- **The price of not knowing:** about 4.6% wider than choosing the best scorer in advance.

Flaw and correction

The first version estimated from stage two alone, treating the pilot only as “known labels”. With no useful scorer it came out 18% *wider* than random sampling (0.147 against 0.124), because it ignored that the pilot is itself a random sample of the whole population. Combining the pilot mean with the stage-two estimate fixed it: with useless scorers it now reproduces random sampling, and with one good scorer it was about 30% narrower than random sampling in synthetic tests.

Verdict: passed, with a caveat. The planner was designed after seeing the same three text datasets it was tested on, so the 6% gain is in-sample; its safety was later checked on a fourth domain (Phase 4). **What was gained:** a scorer-selection step that is safe by construction and pays off when a scorer works; it is best understood as insurance for when you do not know whether a scorer works on your data.

Phase 4: how often does an LLM judge disagree with humans?

Question. The same machinery can audit an LLM used as a judge. Given GPT-4's pairwise verdicts and human expert votes on the same comparisons, how often does the judge disagree with humans, and how many human labels does it take to know? This was also a new, real domain the planner was not designed on.

Step 1: judge reliability on MT-Bench

What we used

- **Data:** the public MT-Bench human judgments (HuggingFace *lmsys/mt_bench_human_judgments*): expert votes and GPT-4 pair verdicts for the same model comparisons. Items are (question, model pair, turn) after putting each pair in a canonical order. The human label is the strict plurality of expert votes (items with tied votes dropped); the item is positive when GPT-4's verdict (first model better, second better, or tie) differs. This left 1,612 items.
- **Categories** were derived from MT-Bench's question numbering, which runs in blocks of ten (81 to 160): writing, roleplay, reasoning, math, coding, extraction, stem, humanities.
- **Cheap scorers** computed with no human label: *closeness* (one minus the gap between the two models' GPT-4 win rates; close contests are harder), *length gap*, *hard category* (reasoning, math, coding), and *turn*.
- **Experiments:** the whole population at 100 and 200 labels (1,000 repetitions), and each category as its own slice at 80 labels with a 24-label pilot (500 repetitions). Pilot picking was made two-sided (a reversed scorer stratifies equally well) and ties in coarse scores are broken at random.
- **Code:** judge_data.py (merge and scorers), run_phase4.py, simulate3.py.

> **Bar fixed in advance:** (1) on the population at 200 labels every arm's coverage within 0.92 to 0.98; (2) on the slices the pilot planner's pooled coverage within 0.92 to 0.98 and no slice below 0.88; (3) the pilot planner never more than 3% wider than random sampling on the population or any slice. The gain over random sampling is reported but not pass/fail; the scorers are weak heuristics, so "no gain and no harm" was an expected outcome.

Results

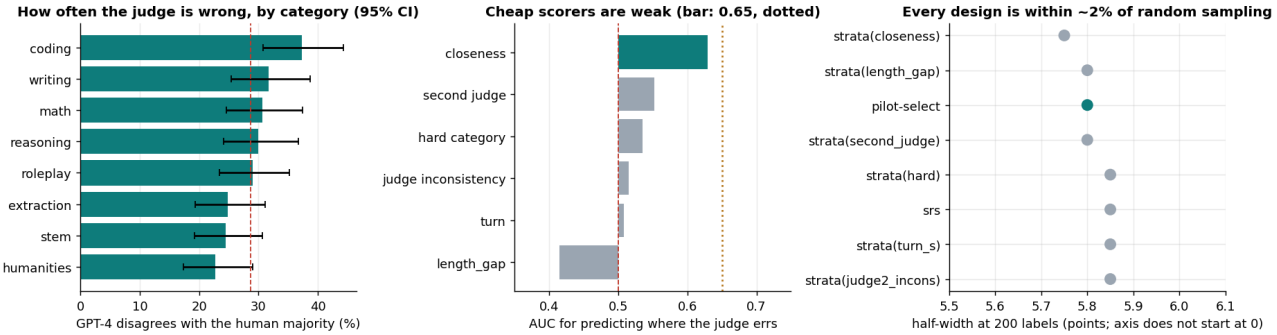


Figure 9. Left: how often GPT-4 disagrees with the human majority, by category, with 95% intervals. Middle: AUC of each cheap scorer for predicting where the judge errs; the dotted line is the 0.65 bar. Right: interval half-width at 200 labels for each design (axis does not start at zero).

Category	Items	GPT-4 disagrees with human majority
coding	193	37.3%
extraction	206	24.8%
humanities	198	22.7%
math	196	30.6%
reasoning	200	30.0%
roleplay	224	29.0%
stem	212	24.5%
writing	183	31.7%
All	1,612	28.7%

- **Validity held.** With the first four scorers coverage was 0.944 to 0.95 on the population at 200 labels; the pilot planner's pooled coverage on the slices was 0.945 (lowest slice 0.934).
- **The judge's error rate is known to about ±5.9 points from 200 human labels** (random-sampling interval), out of 1,612 items: roughly one tenth of the data.
- **Safety held.** The pilot planner was never more than 0.3% wider than random sampling anywhere, and was 1.1% (population) and 1.5% (slices) narrower. Only closeness had any signal (AUC 0.629); length gap pointed the opposite way (0.415), hard category 0.535 and turn 0.508.
- **Categories cannot be ranked at this budget.** At 80 labels a slice interval is about ±7.6 points, but the categories span only 22.7% to 37.3%, and their intervals overlap.

Flaw and correction

A dry run on synthetic data in the same format failed the safety check: on small slices (about 140 items, 80 reviewed) the pilot planner came out 6.7% wider than random sampling, worst slice 8.2%. The two-stage estimator weighted its parts by label counts, which is only right when a small share of the population is sampled. Working through the variance showed count weights cost about 16% more variance at that sampling fraction; weights based on each part's own variance factor restore parity with random sampling and are still exactly unbiased. At Phase 3's scale (200 of 4,000) the two weightings are almost identical.

Step 1b: a second judge as a risk score

Idea. If a second, independent judge disagrees with GPT-4 on a pair, GPT-4 is more likely wrong there. That would give a risk score strong enough for risk-targeted review and narrower intervals.

What we used and did

- **Second judge:** the same open model, Qwen2.5-3B-Instruct on a T4, judging every pair in both answer orders, with the two orders averaged to cancel position bias. Prompts hold two full answers, so the labeler was changed to read only the last position's logits and to halve its batch on a memory error.
- **Risk score:** if GPT-4 said the first model won, risk is one minus the local judge's probability that the first model is better; if GPT-4 said the second won, risk is that probability; if GPT-4 said tie, risk is how strongly the local judge prefers either side. A second score, *judge inconsistency*, measures how much the local judge contradicts itself when the order is swapped.
- **Code:** second_judge.py, run_phase4b.py; 1,612 items scored in both orders (3,224 prompts).

> **Bar fixed in advance:** (1) AUC of the risk score against GPT-4's real errors at least 0.65; (2) error rate among the top 20% by risk at least 1.5 times the base rate; (3) strata on the risk score at least 10% narrower than random sampling on the population (200 labels) and pooled over slices (80 labels); (4) step 1's validity and safety still hold.

Results

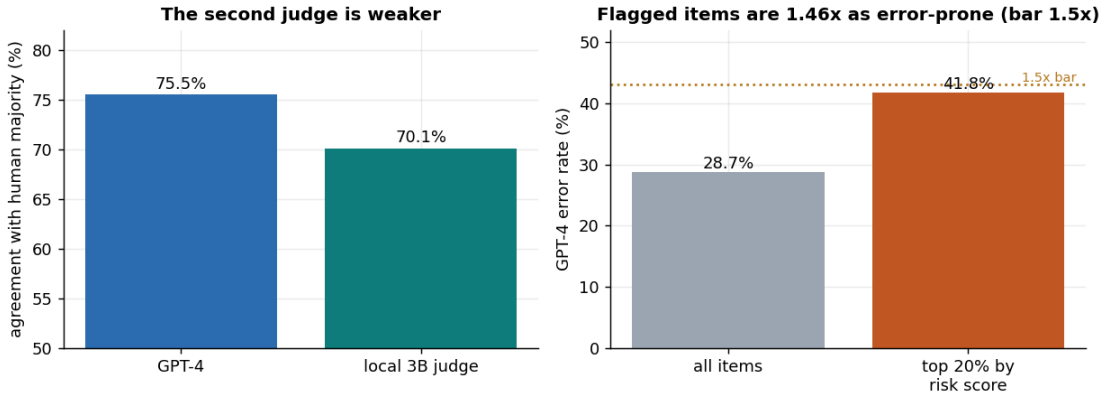


Figure 10. Left: agreement with the human majority on non-tie items. Right: GPT-4's error rate on all items against the top 20% by second-judge risk; the dotted line is the 1.5 times bar.

Check	Bar	Result	Verdict
Risk-score AUC against GPT-4's errors	0.65	0.552 (inconsistency score: 0.515)	fail
Error rate in top 20% by risk	1.5 times base (43.1%)	41.8% against 28.7% base = 1.46 times	fail (narrowly)
Interval narrower than random sampling	10% on population and slices	0.8% on the population, 1.5% on slices	fail
Validity and safety from step 1	coverage 0.92 to 0.98; never 3% wider	coverage 0.933 to 0.958; worst slice +0.1%	pass

Design (200 labels, whole population)	Interval width	Coverage	Typical error
srs	0.117	0.938	0.030
strata(closeness)	0.115	0.939	0.030
strata(length_gap)	0.116	0.947	0.030
strata(hard)	0.117	0.958	0.029
strata(turn_s)	0.117	0.943	0.030
strata(second_judge)	0.116	0.951	0.030
strata(judge2_incons)	0.117	0.933	0.031
pilot-select	0.116	0.940	0.030

Design (100 labels, whole population)	Interval width	Coverage	Typical error
srs	0.171	0.948	0.044
strata(closeness)	0.168	0.948	0.042
strata(length_gap)	0.170	0.934	0.044
strata(hard)	0.172	0.943	0.045
strata(turn_s)	0.171	0.945	0.044
strata(second_judge)	0.170	0.944	0.044
strata(judge2_incons)	0.172	0.947	0.044
pilot-select	0.170	0.944	0.043

Category slice (80 labels)	pilot-select	srs	strata (closeness)	strata (second_judge)	strata (length_gap)
coding	0.162	0.163	0.158	0.161	0.160
extraction	0.148	0.148	0.149	0.148	0.149
humanities	0.140	0.141	0.136	0.141	0.139
math	0.155	0.156	0.156	0.153	0.155
reasoning	0.156	0.156	0.157	0.153	0.155
roleplay	0.158	0.160	0.153	0.159	0.159
stem	0.143	0.149	0.133	0.148	0.146
writing	0.148	0.154	0.151	0.143	0.144

Interval widths by category slice for the main designs (pooled coverage 0.944 to 0.948 for every design).

Why it failed, and it is not a bug. The local judge agrees with humans 70.1% of the time against GPT-4's 75.5%. When a weaker judge disagrees with a stronger one, the weaker is more often the one that is wrong. If the two judges' errors were independent, the expected GPT-4 error rate among disagreements is 0.245 × 0.701 / (0.245 × 0.701 + 0.755 × 0.299) = 43%; the observed rate was 41.8%. The second judge behaved exactly as theory predicts for its quality.

> **Verdict: failed all three new checks; validity and safety held. What was gained:** a measured, mechanistic reason why a weaker judge cannot pinpoint a stronger one's mistakes; a modest real signal (the top 20% by risk hold 46% more GPT-4 errors per review than random review); and the decision to park risk-targeted review and the learned-escalation comparison until a stronger or more independent risk signal exists.

Phase 5: can we monitor continuously?

Question. Teams audit a few items every week and check the dashboard after each batch. An ordinary 95% interval is valid for one look at a pre-chosen sample size; checking it after every label makes it miss far more than 5% of the time. Can we build an interval and a drift alarm that stay valid however often you look?

What we used and did

- **Method:** a betting confidence sequence for a yes/no outcome (“the judge was wrong”). For each candidate error rate two gamblers bet that the true rate is above or below it, with bets chosen only from past data; under the true rate their capital is a nonnegative martingale, so by Ville’s inequality the true rate stays in the set of surviving candidates at every time simultaneously, except with probability 5%. The two gamblers are averaged, a grid of 400 candidate rates is used, the grid is widened by one step, and the sets are intersected over time.
- **Drift alarm:** fires the first time the confidence sequence’s lower bound exceeds the error rate the team agreed to accept, from the tenth audited item on.
- **Data:** streams are bootstrap draws from the real MT-Bench judge-error outcomes (28.7%). The real data has no time order, so shifts are constructed: to the coding category (37.3%, real) and to synthetic 45% and 55%.
- **Experiments:** validity and false alarms at three true error rates (the real 28.7%, 10%, 50%) with 1,000 streams of 1,000 labels each; detection after a shift with 300 labels at the baseline then 1,500 after, 500 streams per scenario. Every number was also computed for ordinary Wilson intervals recomputed after every label.
- **Code:** monitor.py and run_phase5.py (CPU, about two minutes).

> **Bar fixed in advance:** (1) the confidence sequence’s miss rate (truth outside it at any look from the tenth label) at most 6% at each of the three true error rates; (2) with no shift, the alarm fires within 1,000 labels at most 6% of the time. Reported, not pass/fail: the same numbers for ordinary intervals, detection delay, and the width price.

Results

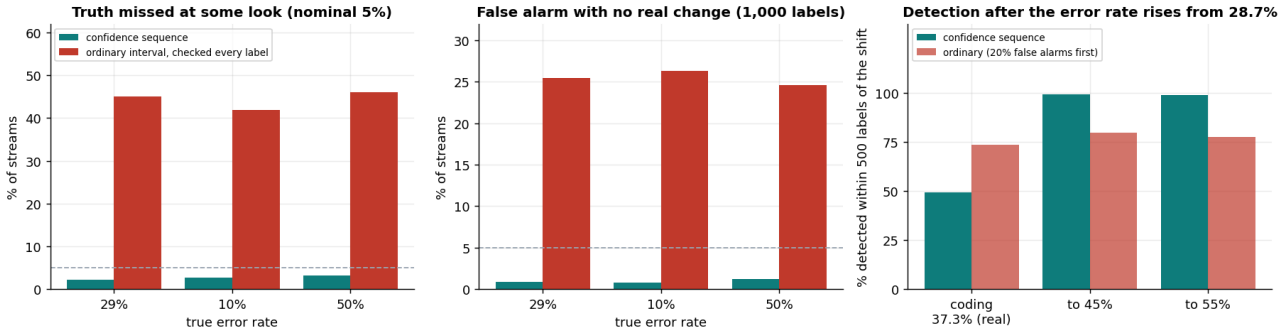


Figure 11. Left: share of streams where the truth was missed at some look (target 5%, dashed). Middle: false alarms with no real change. Right: detection within 500 labels after the error rate rises from 28.7%; the ordinary interval's apparent speed comes with about 20% false alarms before the shift.

True error rate	Confidence sequence: missed at some look	Ordinary interval, checked every label	Confidence sequence: false alarm	Ordinary: false alarm	Width vs ordinary at 100 / 200 / 500 labels
28.7% (real)	2.2%	45.0%	0.9%	25.5%	1.53 / 1.56 / 1.62 times
10% (synthetic)	2.6%	41.9%	0.8%	26.3%	1.51 / 1.57 / 1.70 times
50% (synthetic)	3.2%	46.0%	1.2%	24.6%	1.48 / 1.51 / 1.58 times

Shift from 28.7% to	Method	False alarm before the shift	Detected within 250 labels	Within 500	Within 1,000	Median delay (labels)
coding category (real, 37.3%)	confidence sequence	0.6%	9.8%	49.2%	94.8%	505
coding category (real, 37.3%)	ordinary, every look	20.8%	40.2%	73.6%	79.2%	246
synthetic 45%	confidence sequence	1.0%	67.4%	99.0%	99.0%	214
synthetic 45%	ordinary, every look	20.4%	77.0%	79.6%	79.6%	115
synthetic 55%	confidence sequence	1.4%	98.6%	98.6%	98.6%	123
synthetic 55%	ordinary, every look	22.4%	77.6%	77.6%	77.6%	71

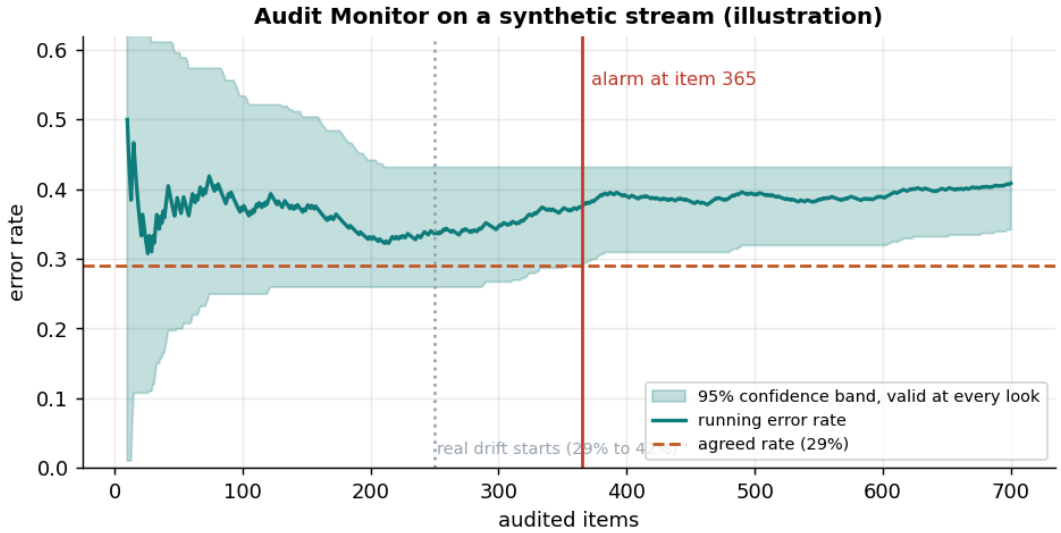


Figure 12. An illustration on a synthetic stream: the band is the confidence sequence, the line the running error rate, the dashed line the agreed rate. The error rate truly rises from 29% to 42% at item 250; the alarm fires at item 365.

- **Validity:** the confidence sequence missed 2.2% to 3.2% of streams against 41.9% to 46.0% for ordinary intervals; with no change it false-alarmed about 1%, against 25% for ordinary intervals.
- **The price is width and speed:** about 1.5 to 1.6 times an ordinary interval. A real 8.6-point rise (the coding category) took about 500 labels to flag at the median (49% within 500, 95% within 1,000); a 16-point rise about 214; a 26-point rise about 123. A team auditing 20 items a week would need roughly half a year to flag a 9-point rise at the median, so catching moderate drift sooner means auditing more items.
- **The ordinary interval looks faster only because it cries wolf:** about 20% of its streams false-alarmed before any real shift.

Verdict: passed both checks. Limits: the streams are bootstrap draws and most shifts are synthetic; the method assumes audited items are independent random draws from the stream, each audited with a fixed known probability. **What was gained:** an interval and alarm that can be watched continuously without breaking their guarantee, with measured costs.

Tools and artifacts built

Artifact	What it does	Status and caveats
BotShare (deployable API and page)	Upload a CSV of account profiles; get an estimated bot share with an interval, a distribution-shift warning, a full and a core model, and a reliability rule; it returns aggregates only	Trained on the real datasets. Both profiles failed the reliability bar, so the page withholds its headline number. A scikit-learn version mismatch (the model is a pickle) is now handled by pinning in the container. Not deployed.
Check My Detector (browser tool)	Load a scored CSV, label a random pilot, let the pilot pick a scorer, label a stratified second stage, get a prevalence interval and a certificate; skipping the pilot adds precision and recall	Runs in the page, uploads nothing. Prevalence intervals validated in simulation; recall interval and the pilot design on new data not validated. Weights updated for small populations. The certificate flags “check your labels” when the pilot cannot judge any score.
Audit Monitor (browser tool)	Paste a running 0/1 audit log and an agreed error rate; see the always-valid interval over time and a drift alarm	Validated in simulation on bootstrap streams; real time drift not tested.
solar package	Audit, planners, estimators, simulations for Phases 1 to 5, LLM and judge pipelines	33 unit and end-to-end tests pass; the real runs used Kaggle with scikit-learn 1.6.1 and Qwen2.5-3B-Instruct.
Results report (HTML and notebook)	Rebuilds every figure and table from saved CSVs or a built-in snapshot	Some sections always use the snapshot (listed in its README).
Write-up (Claude Doc) and this PDF	Narrative documentation of the project	Captions describe the original runs.

Flaws found along the way and how they were fixed

Most of the project’s value is in what was caught. Each item below was found by a test, an audit or a failed bar, not by luck.

#	What went wrong	How it was found	Effect and fix
1	Original notebook claims unsupported (fusion adds nothing, four agents inert, LLM agent never ran, compute claim unmeasured)	ablation and output audit	Project reframed around evaluation
2	cresci-2015’s bots are all inside cresci-2017	account-id overlap check	Cross-dataset AUC for cresci-2015 corrected from 0.986 to 0.877; shared accounts now removed from training
3	FakeNewsNet articles under both labels (114 of about 289)	duplicate and conflict check	Dataset cut to 175 articles and demoted to a small check
4	Reliability rule too lenient: a one-point edge on 4 of 7 datasets counted as reliable	per-dataset recomputation of a printed result	Rule now needs a 25% lower error and wins on 70% of datasets
5	Model could not be loaded under a different scikit-learn version	clean-environment replay of the container steps	Training writes its version; the container pins to it
6	Expectation that PPI always narrows intervals was wrong	a failing test and its diagnostic	Learned that detector information goes in the design or the estimator, not both
7	Two-stage estimator 18% wider than random sampling with useless scorers	a failing safety test	Pilot mean combined with the stage-two estimate
8	Count-based weights cost about 7% width on small slices	the safety check on synthetic slices	Weights from each part’s variance factor; parity restored
9	Verdict crashed when the bar’s 200-label budget was not simulated	a smoke-test command	Falls back to the largest budget and marks the verdict indicative only
10	Certificate said “certified” on a log where nearly every item was marked bot (93%)	a screenshot from a test run	New verdict “check your labels” and an extreme-share flag
11	Charts with truncated bar axes, legends covering bars, a wrong reference line	looking at the rendered figures	Bars start at the 0.5 chance line or use dot plots; legends moved; line recomputed
12	Long judge prompts would exhaust GPU memory	estimating tensor sizes before the run	Last-position logits only; batch halving on memory errors (untested on a GPU)

Scorecard and lessons

Component	Verdict	Evidence
Audit checks (overlap, duplicates, shortcuts)	earned its place	found 3,351 shared accounts, label-conflict duplicates, 22% duplicate rows in one dataset
Labeled-sample estimator	earned its place	coverage 0.94-0.96 on 7 bot and 3 text datasets; +/-5.4 pts at 200 labels vs 29 pts error from counting flags
Equal-strata planner	modest	about 9% narrower than random sampling with weak detectors
PPI on top of strata	did not earn it	0.3-0.4% gain with strata, 7% only with random sampling
Detector-based bot-share estimators	failed	no estimator beat guessing the average; stable features and simpler models did not rescue it
LLM labeler (3B, general)	failed the bar	5.9% pooled vs 15% required; helps only where its AUC is high (CoAID 0.87)
Definition agent v0 (rubric prompt)	failed the bar	width change -0.2%; hurt one dataset by 0.14 AUC
Pilot-then-stratify planner	earned its place (in-sample)	~6% narrower, never worse than random sampling; confirmation on new data still limited
Judge reliability on MT-Bench	validity confirmed	coverage 0.944-0.95; GPT-4 disagrees with humans on 28.7%; planner gain ~1% (weak scorers)
Second-judge risk score	failed the bar	AUC 0.552 vs 0.65; top-20% lift 1.46x vs 1.5x
Continuous monitoring (confidence sequence)	earned its place	miss rate 2-3% vs 42-46% for ordinary intervals checked every label; ~1.5x wider

Lessons

- **Measure on data the system has never seen.** Within-dataset numbers flatter every detector and every model.
- **Fix the success bar before the run.** It stopped near-misses being rounded into passes (a 1.46 times lift against a 1.5 times bar) and let negative results stand.
- **Sample size, not model quality, drives accuracy.** A few hundred random human labels beat every automatic estimate tried.
- **Use automatic scores only through a human-labeled correction.** Then a bad scorer costs interval width, never validity.
- **Scorer quality is dataset-specific.** Measure it on a small pilot instead of assuming it.
- **Anything watched repeatedly needs an interval built for repeated looks.**
- **Most bugs were found by testing the safe-fallback behaviour and by looking at rendered output.**
- **Negative results decided what to build next** at every stage; they are part of the product, not noise around it.

Limitations and threats to validity

- **Labelers are assumed correct.** Every simulation treats a dataset's labels as the human reviewer. Real reviewers err, which would bias every estimate.
- **The pilot planner is not independently confirmed.** It was designed after seeing the three text datasets it was tested on; the gain (about 6%) is in-sample, though its safety held on MT-Bench.
- **Monitoring evidence is simulated.** The real data has no time order; streams are bootstrap draws and most shifts are constructed. The method assumes independent random draws from the stream.
- **Models and features are small.** Bot detectors use 13 profile fields; the labeler is a 3B model, and a larger one might do better. Risk-targeted review needs a stronger signal than the cheap scorers gave.
- **"Bot" and "fake" mean different things per dataset.** Part of the transfer failure is that labels describe different kinds of account or claim, not only that models are fragile.
- **Several figures come from one run.** Single real runs with fixed seeds; repetition counts are large, but there is no run-to-run replication of the LLM scoring.
- **Profile fields only.** No tweets or graphs for most bot datasets, so no result here says anything about content-based detection.
- **Related-work claims are partial.** Papers were found by search and their abstracts read; none was read in full.

Related work

- **Bot detection that generalizes:** an ensemble of specialized classifiers for novel bots (Sayyadiharikandeh et al., CIKM 2020, arxiv.org/abs/2006.06867); simple models reach near state of the art and benchmarks generalize poorly (Hays et al., 2023, arxiv.org/pdf/2301.07015); detector-based bot counts are unreliable (Rauchfleisch and Kaiser, 2020, PMC7580919).
- **Valid evaluation with few human labels:** ARES, prediction-powered inference for RAG evaluation (arxiv.org/pdf/2311.09476); a 2026 paper on allocating a human-validation budget with prediction-powered inference (arxiv.org/pdf/2604.12497); efficient inference for noisy LLM judges (arxiv.org/html/2601.05420v1).
- **Human review in moderation:** uncertainty-based review under limited reviewers (arxiv.org/pdf/2107.04212); learning to defer (arxiv.org/pdf/2402.12237v4).
- **From memory, not checked here:** prediction-powered inference (Angelopoulos et al., 2023) and betting confidence sequences (Waudby-Smith and Ramdas).
- **What is new, to be confirmed against these papers:** the overlap and conflicting-duplicate audit across seven bot benchmarks, the failure of standard share-correction methods across them, and the pilot-based choice among scorers. The individual statistical tools are not new.

Reproducibility

Three code folders hold everything: *botshare* (features, bot-share estimators, the rescue experiment, a deployable API), *solar* (audit, planners, estimators, Phases 1 to 5, LLM and judge pipelines, 33 tests) and *solar_report* (the report generator). Run on Kaggle in this order; a T4 GPU and Internet are needed only for the LLM steps.

Step	Command (paths shown for Kaggle)	Needs
Baselines and audit	run_baselines('/kaggle/input/datasets', cache_dir='/kaggle/working/cache') (solar_phase1.py)	CPU
Bot-share estimation	python train.py --root /kaggle/input/datasets --out /kaggle/working/model (botshare)	CPU
Rescue attempt	python transfer_experiment.py --root /kaggle/input/datasets --out /kaggle/working/transfer (botshare)	CPU
Phase 1	python run_phase1.py --root /kaggle/input/datasets --out /kaggle/working/phase1 (solar)	CPU
Phase 2	python run_phase2.py --root /kaggle/input/datasets --out /kaggle/working/phase2 --backend hf --pop 4000	GPU, Internet
Phase 3	python run_phase3.py --root /kaggle/input/datasets --phase2-out /kaggle/working/phase2 --out /kaggle/working/phase3	CPU
Phase 4	python run_phase4.py --out /kaggle/working/phase4	Internet
Phase 4b	python run_phase4b.py --out /kaggle/working/phase4b	GPU, Internet
Phase 5	python run_phase5.py --items /kaggle/working/phase4b/items_scored.csv --out /kaggle/working/phase5	CPU
Results report	python make_report.py --root /kaggle/working --out /kaggle/working/solar_report	CPU

Each script’s docstring holds the bar used to judge its result, so a re-run can be judged against the same criteria. The tests run with *pytest tests* in the solar folder.

Glossary

Term	Meaning
AUC	Chance that a random positive scores above a random negative; 0.5 is chance, 1.0 perfect
Coverage	How often a 95% interval contains the truth over repeated samples
Stratified sampling	Split items into groups (here five groups by score rank) and label a set number from each
Finite-population correction	Shrinks the interval when a large share of a small population is labeled
PPI (prediction-powered inference)	Uses a model's predictions on every item, corrected by human labels on a sample, so the estimate stays unbiased for any model
Pilot	A small random sample used to measure which scorer works before spending the rest of the budget
Quantification	Estimating the share of a class in a group rather than labeling each item
Leave-one-dataset-out	Train on all datasets but one and test on the one left out
Confidence sequence	An interval valid at every time at once, so it can be checked after every observation
Ville's inequality	The result that gives a nonnegative martingale a bounded chance of ever exceeding a threshold
Drift alarm	Fires when the interval's lower bound rises above the agreed error rate
Oracle (benchmark)	The best choice made with knowledge no real user would have

Appendix A. Bot-share estimates per dataset

Profile	Held-out dataset	True share	Naive count	Average score	Adjusted count
full	botometer-feedback-2019	26.6%	21.6%	30.5%	31.3%
full	cresci-2015	63.2%	44.3%	47.4%	100.0%
full	cresci-2017	75.8%	31.4%	37.9%	100.0%
full	cresci-rtbust-2019	50.9%	16.5%	27.7%	13.5%
full	cresci-stock-2018	53.5%	41.7%	44.3%	100.0%
full	gilani-2017	43.9%	5.9%	13.0%	0.0%
full	midterm-2018	84.0%	34.4%	40.3%	100.0%
core	botometer-feedback-2019	26.6%	23.6%	33.0%	30.2%
core	cresci-2015	63.2%	65.2%	62.5%	100.0%
core	cresci-2017	75.8%	56.6%	55.1%	100.0%
core	cresci-rtbust-2019	50.9%	13.3%	22.9%	0.8%
core	cresci-stock-2018	53.5%	41.6%	43.7%	78.5%
core	gilani-2017	43.9%	4.0%	13.4%	0.0%
core	midterm-2018	84.0%	65.9%	61.9%	100.0%

Appendix B. Text transfer results

Trained on	Tested on	Kind	AUC	Macro F1 at threshold 0.5
fake-and-real-news	fake-and-real-news	within	0.999	0.987
liar	liar	within	0.675	0.634
coaid	coaid	within	0.977	0.891
coaid-news	coaid-news	within	0.978	0.889
fake-and-real-news	liar	cross	0.544	0.494
fake-and-real-news	fakenewsnet	cross	0.752	0.444
fake-and-real-news	coaid	cross	0.576	0.272
fake-and-real-news	coaid-news	cross	0.585	0.287
liar	fake-and-real-news	cross	0.648	0.608
liar	fakenewsnet	cross	0.618	0.624
liar	coaid	cross	0.514	0.429
liar	coaid-news	cross	0.520	0.439
coaid	fake-and-real-news	cross	0.583	0.422
coaid	liar	cross	0.540	0.388
coaid	fakenewsnet	cross	0.471	0.400

The low macro F1 values across datasets (for example 0.272 for Fake and Real News tested on CoAID despite AUC 0.576) show that decision thresholds do not transfer either: a model trained on one dataset labels almost everything as one class on another.

Appendix C. Single-feature AUC for every feature and dataset

Feature	botometer	cresci-15	cresci-17	rtbust	stock-18	gilani	midterm
log_followers	0.58	0.09	0.18	0.36	0.27	0.50	0.03
log_following	0.54	0.63	0.38	0.32	0.34	0.53	0.07
log_tweets	0.56	0.06	0.07	0.38	0.44	0.75	0.04
log_listed	0.51	0.24	0.29	0.33	0.36	0.56	0.10
log_ff_ratio	0.58	0.01	0.16	0.53	0.43	0.48	0.40
verified	0.47	0.50	0.50	0.50	0.49	0.46	0.47
has_desc	0.48	0.38	0.31	0.24	0.38	0.51	0.22
desc_len	0.48	0.36	0.35	0.22	0.37	0.56	0.16
has_url	0.47	0.34	0.43	0.44	0.44	0.55	0.31
has_location	0.42	0.58	0.40	0.34	0.46	0.50	0.50
name_len	0.56	0.45	0.68	0.43	0.49	0.52	0.40
username_len	0.56	0.62	0.64	0.54	0.52	0.46	0.74
username_digit_ratio	0.58	0.48	0.47	0.52	0.56	0.52	0.80

Below 0.5, bots have lower values than humans; above 0.5, higher. The same feature changes direction between datasets.